

UNIVERSITÉ DON BOSCO DE LUBUMBASHI

INTRODUCTION GÉNÉRALE

1. Problématique

La gestion des Travaux de Fin de Cycle (TFC) constitue un processus académique central au sein des universités congolaises en général, et à l'Université Don Bosco de Lubumbashi (UDBL) en particulier. Ce processus englobe plusieurs étapes critiques : le dépôt d'une fiche de sujet, la décision académique, l'assignation d'un directeur de mémoire, l'encadrement, le dépôt du mémoire, l'autorisation de soutenance et l'archivage.

Actuellement, ce processus repose principalement sur des mécanismes manuels et fragmentés (courriels, échanges verbaux, dossiers papier), ce qui entraîne des retards, une absence de traçabilité et un risque de perte documentaire. De plus, l'irruption des outils d'intelligence artificielle générative soulève la question de l'intégrité scientifique des travaux soumis, alors qu'aucun contrôle automatisé n'est en place.

Face à ces constats, la question centrale de ce mémoire est : **comment concevoir et déployer une plateforme web qui automatise la gestion des TFC, garantit la traçabilité des décisions et intègre un module de détection du contenu généré par IA ?**

2. Hypothèse

Nous partons de l'hypothèse que la conception et le déploiement d'une plateforme web intégrée, associée à des modules d'analyse automatisée (détection IA, extraction de texte PDF) et à un workflow de notifications, permettront :

- de réduire significativement les délais de traitement ;
- d'assurer une traçabilité et un archivage consultable des décisions et des documents ;
- d'alerter automatiquement en cas de suspicion de contenu généré par IA ;
- d'améliorer la qualité de l'encadrement et la transparence du processus.

3. État de l'art

Les systèmes de gestion des mémoires et des dépôts institutionnels existent sous diverses formes : répertoires institutionnels, modules de gestion de thèses et fonctionnalités LMS (Learning Management System). De nombreux établissements combinent des solutions open-source et propriétaires pour gérer les dépôts, le suivi et l'archivage.

En matière de détection d'originalité et de plagiat, des outils établis (par ex. Turnitin, Unicheck) analysent les similarités textuelles par rapport à des bases documentaires. Plus récemment, des services spécialisés dans la détection de contenu généré par IA (ex. GPTZero et autres) proposent des

scores probabilistes basés sur des métriques linguistiques (perplexité, entropie, patterns de génération). Toutefois, ces services présentent des limites : coût, couverture linguistique variable, faux positifs/négatifs, et intégration non systématique dans les workflows universitaires.

L'analyse de l'état de l'art montre un besoin d'intégration : relier la soumission, l'extraction automatique de texte (PDF), l'évaluation par des API de détection IA et le workflow décisionnel (validation/rejet/notification). C'est précisément cette intégration end-to-end que vise notre contribution.

4. Choix et intérêt du sujet

4.1 Choix du sujet

Le choix de porter l'étude sur une plateforme de gestion des TFC s'explique par trois raisons principales : la problématique locale identifiée à l'UDBL (digitalisation incomplète), l'importance institutionnelle d'assurer l'intégrité académique et la pertinence pédagogique du projet pour démontrer des pratiques d'ingénierie logicielle modernes (architecture MVC, bonnes pratiques de sécurité, tests). Le sujet permet d'articuler des enjeux techniques, organisationnels et éthiques.

4.2 Intérêt du sujet

- **Intérêt scientifique** : le projet met en application des concepts du génie logiciel (architecture, modélisation, tests) et explore l'intégration de modules d'intelligence artificielle pour la garantie d'intégrité documentaire.
- **Intérêt pratique** : il répond à un besoin opérationnel réel à l'UDBL : réduction des délais, traçabilité, archivage numérisé et détection automatisée de contenus suspectés d'être générés par IA.
- **Intérêt pédagogique / personnel** : ce travail permet aux auteurs d'appliquer et d'approfondir des compétences en développement web, en gestion de bases de données, en modélisation UML et en intégration d'API tierces.

5. Approche méthodologique

L'approche adoptée combine modélisation, développement itératif et validation terrain :

- **Conception** : modélisation des besoins par UML (cas d'utilisation, diagrammes de classes et de séquence) pour formaliser les processus et les acteurs ;
- **Développement** : mise en œuvre d'une application web en architecture MVC (Laravel), développement itératif en méthode agile avec intégration continue des retours ;
- **Intégration d'API** : extraction de texte depuis les PDF (bibliothèques de parsing) puis envoi aux services de détection IA (API) pour génération de rapports automatisés ;
- **Collecte des besoins** : interviews et observations auprès des acteurs (secrétariats, enseignants, étudiants) pour ajuster les fonctionnalités ;
- **Validation** : tests unitaires et fonctionnels (PHPUnit), déploiement pilote et retours d'usage pour mesurer l'impact sur les délais et la traçabilité.

6. Délimitation du sujet

Délimitation spatiale

L'étude est limitée à l'Université Don Bosco de Lubumbashi et à ses quatre facultés (ESIS, ECOPO, KANSEBULA, THEOLOGICUM).

Délimitation temporelle

Les données et cas pratiques considérés portent principalement sur l'année académique 2025–2026. La solution reste toutefois conçue pour être extensible à d'autres années.

Délimitation fonctionnelle

Le périmètre couvre : gestion des utilisateurs et rôles (RBAC), soumission/validation des sujets, dépôt des fichiers PDF, extraction et analyse automatique des documents, notifications, archivage et administration. Le projet **ne couvre pas** la composition du jury ni la gestion des notes ni la gestion financière.

7. Subdivision du travail

Le mémoire est structuré comme suit :

- **Chapitre I** : Considérations théoriques et analyse du système existant ;
- **Chapitre II** : Conception et modélisation (UML, modèle de données) ;
- **Chapitre III** : Implémentation, tests et résultats (architecture technique, interfaces, validation) ;
- **Annexes** : guides d'installation, extraits de code, rapports d'analyse IA, captures d'écran.

8. Outils logiciels et équipements utilisés

Le projet a été mis en œuvre avec la pile technologique suivante (résumé) :

- **Langage & framework** : PHP 8.2+, Laravel 12 ;
- **Base de données** : PostgreSQL (production), SQLite (tests) ;
- **Frontend & build** : Tailwind CSS, Vite, Node.js ;
- **Gestion des dépendances** : Composer (PHP), npm/yarn (JS) ;
- **Packages clés** : spatie/laravel-permission (RBAC), smalot/pdfparser (extraction PDF) ;
- **Détection IA** : intégration d'une API (ex. GPTZero) via un service (voir `app/Services/AiDetectionService.php`) ;
- **Tests & qualité** : PHPUnit, laravel/pint ;
- **Infrastructure & outils dev** : Git, Docker (optionnel), serveur PHP local pour le développement ;
- **Matériel requis** : poste de développement standard (4+ CPU, 8+ GB RAM recommandé), connexion Internet pour les appels API.

Pour un inventaire détaillé et des preuves (fichiers de configuration, composer.json, package.json), voir le document dédié : [docs/OUTILS-ET-EQUIPEMENTS.md](#).

LISTE DES FIGURES

N°	Figure	Page
1	Organigramme de l'Université Don Bosco de Lubumbashi	—
2	Diagramme de cas d'utilisation global	—
3	Diagramme de cas d'utilisation — Étudiant	—
4	Diagramme de cas d'utilisation — Chef de Département	—
5	Diagramme de cas d'utilisation — Enseignant	—
6	Diagramme de cas d'utilisation — Administrateur	—
7	Diagramme de séquence — Soumission de sujet	—
8	Diagramme de séquence — Validation/Rejet de sujet	—
9	Diagramme de séquence — Dépôt de fichier TFC avec analyse IA	—
10	Diagramme de séquence — Autorisation de soutenance	—
11	Diagramme de classes	—
12	Modèle relationnel de la base de données	—
13	Diagramme d'activités — Processus complet de gestion d'un TFC	—
14	Architecture MVC de Laravel	—
15	Capture d'écran — Page d'accueil	—
16	Capture d'écran — Formulaire de connexion	—
17	Capture d'écran — Tableau de bord Administrateur	—
18	Capture d'écran — Tableau de bord Étudiant	—
19	Capture d'écran — Formulaire de soumission de sujet (5 étapes)	—
20	Capture d'écran — Tableau de bord Chef de Département	—
21	Capture d'écran — Tableau de bord Enseignant	—
22	Capture d'écran — Rapport d'analyse IA	—
23	Capture d'écran — Archives publiques	—
24	Capture d'écran — Gestion des utilisateurs	—
25	Capture d'écran — Gestion des facultés et filières	—

LISTE DES TABLEAUX

N°	Tableau	Page
1	Comparaison entre le système existant et le système proposé	—
2	Tableau des acteurs du système	—
3	Description des cas d'utilisation	—
4	Dictionnaire de données	—
5	Structure de la table <code>users</code>	—
6	Structure de la table <code>faculties</code>	—
7	Structure de la table <code>departments</code>	—
8	Structure de la table <code>subjects</code>	—
9	Structure de la table <code>thesis_files</code>	—
10	Structure de la table <code>ai_reports</code>	—
11	Structure de la table <code>academic_years</code>	—
12	Structure de la table <code>activity_logs</code>	—
13	Structure de la table <code>system_settings</code>	—
14	Matrice rôles-permissions	—
15	Grille d'interprétation des scores IA	—
16	Technologies et outils utilisés	—
17	Résultats des tests fonctionnels	—

LISTE DES ABRÉVIATIONS

Abréviation	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PDF	Portable Document Format
PHP	PHP Hypertext Preprocessor
PK	Primary Key
RBAC	Role-Based Access Control
SI	Système d'Information
SQL	Structured Query Language
TFC	Travail de Fin de Cycle
UDBL	Université Don Bosco de Lubumbashi
UML	Unified Modeling Language
UUID	Universally Unique Identifier

INTRODUCTION GÉNÉRALE

1. Problématique

La gestion des Travaux de Fin de Cycle (TFC) constitue un processus académique central au sein des universités congolaises en général, et à l'Université Don Bosco de Lubumbashi (UDBL) en particulier. Ce processus englobe plusieurs étapes critiques : le dépôt d'une fiche de sujet, la réponse académique, l'assignation d'un directeur de mémoire, l'encadrement, le dépôt du mémoire, l'autorisation de soutenance et l'archivage.

À l'heure actuelle, l'ensemble de ce processus repose sur des mécanismes majoritairement manuels, fragmentés et faiblement traçables. L'étudiant remplit une fiche qu'il transmet par mail, puis attend une réponse également communiquée par mail. Le directeur de mémoire est désigné par le chef de département et informé verbalement, par téléphone ou parfois par mail. La rédaction et le suivi se font principalement en présentiel. Une fois achevé, le mémoire est imprimé puis déposé à l'administration pour vérification manuelle par le comité de censure, sans contrôle automatisé de plagiat ni de contenu généré par IA. Après accord verbal ou par mail du directeur pour la soutenance, les mémoires sont finalement archivés dans des armoires physiques au secrétariat, avec un accès limité et sans mécanisme de recherche structuré.

Cette organisation présente de nombreuses insuffisances :

- **La lenteur du processus de validation** : les propositions de sujet peuvent rester en attente pendant des semaines faute de mécanisme de notification automatique auprès des responsables ;
- **L'absence de traçabilité** : il n'existe aucun historique centralisé des décisions prises (validations, rejets, motifs), ce qui rend difficile tout audit ou suivi rétrospectif ;
- **Le risque de perte de documents** : les versions papier sont sujettes à la détérioration, à l'égarement ou à la duplication non contrôlée ;
- **L'impossibilité de détecter le plagiat et le contenu généré par IA** : avec la démocratisation des outils d'intelligence artificielle générative (ChatGPT, Gemini, Claude, etc.), le risque que des étudiants soumettent des travaux partiellement ou intégralement générés par ces outils est devenu une préoccupation majeure. Or, aucun mécanisme de vérification n'est actuellement en place ;
- **La difficulté d'accès aux archives** : les travaux des promotions précédentes ne sont pas facilement consultables pour servir de référence ou éviter les doublons de sujets ;
- **L'inefficacité de la communication** : les échanges entre étudiants, enseignants et responsables académiques sont fragmentés et souvent non documentés.

Face à ce constat, la question fondamentale qui guide notre recherche est la suivante : **comment concevoir et implémenter une plateforme web intégrée qui automatise et sécurise l'ensemble du processus de gestion des TFC, tout en intégrant un mécanisme de détection de contenu généré par intelligence artificielle ?**

2. Hypothèses

En réponse à la problématique énoncée, nous formulons les hypothèses suivantes :

1. **La mise en place d'une plateforme web centralisée** permettrait de numériser et d'automatiser le processus de gestion des TFC, depuis la soumission du sujet jusqu'à l'autorisation puis à la planification de la soutenance, en offrant à chaque acteur (étudiant, enseignant, chef de département, administrateur) un espace dédié et des fonctionnalités adaptées à son rôle.
 2. **L'intégration d'un module de détection de contenu IA**, basé sur des API spécialisées telles que GPTZero, permettrait d'évaluer automatiquement le degré d'originalité des mémoires déposés et de signaler les travaux présentant un risque élevé de contenu généré artificiellement.
 3. **Un système de notifications automatisées** (par e-mail et en temps réel dans l'application) améliorerait significativement la réactivité des différents intervenants et réduirait les délais de traitement.
 4. **La mise à disposition d'archives publiques** des travaux défendus favoriserait la consultation des travaux antérieurs et contribuerait à limiter la redondance des sujets de recherche.
-

CHAPITRE PREMIER : CONSIDÉRATIONS THÉORIQUES ET ANALYSE DU SYSTÈME EXISTANT

Section 1 : Définition des concepts fondamentaux

1.1. Système d'information

Un **système d'information** (SI) est un ensemble organisé de ressources (matérielles, logicielles, personnelles, données, procédures) permettant de collecter, stocker, traiter et diffuser l'information au sein d'une organisation. Selon Robert Reix, « un système d'information est un ensemble de composants inter-reliés qui recueillent, traitent, stockent et diffusent l'information pour soutenir la prise de décisions, la coordination et le contrôle d'une organisation » (Reix, 2004).

Dans le contexte universitaire, le système d'information académique englobe la gestion des inscriptions, des cours, des évaluations, et — dans notre cas — la gestion des travaux de fin de cycle.

1.2. Application web

Une **application web** est un logiciel applicatif hébergé sur un serveur distant et accessible via un navigateur web à travers le protocole HTTP/HTTPS. Contrairement aux applications de bureau, les applications web ne nécessitent aucune installation sur le poste client et offrent l'avantage d'être accessibles depuis tout appareil connecté à Internet.

Les applications web modernes suivent généralement une architecture **client-serveur** où :

- Le **client** (navigateur) envoie des requêtes HTTP ;
- Le **serveur** traite ces requêtes, interagit avec la base de données et renvoie des réponses (HTML, JSON, fichiers).

1.3. Architecture Modèle-Vue-Contrôleur (MVC)

L'architecture **MVC** est un patron de conception logicielle qui sépare une application en trois composants interconnectés :

- **Modèle (Model)** : représente les données et la logique métier. Il interagit directement avec la base de données et définit les règles de validation et les relations entre les entités ;
- **Vue (View)** : gère la présentation et l'affichage des données à l'utilisateur. Elle reçoit les données du contrôleur et les formate en interface graphique (HTML/CSS) ;
- **Contrôleur (Controller)** : fait office d'intermédiaire entre le modèle et la vue. Il reçoit les requêtes de l'utilisateur, interroge le modèle approprié et sélectionne la vue à afficher.

Cette séparation des responsabilités favorise la maintenabilité, la testabilité et la réutilisabilité du code.

1.4. Framework

Un **framework** est un cadre de travail logiciel qui fournit une structure de base, des composants réutilisables et des conventions pour accélérer le développement d'applications. Il impose une organisation du code et fournit des outils prêts à l'emploi pour les tâches courantes (routage, authentification, gestion de base de données, etc.).

1.5. ORM (Object-Relational Mapping)

L'**ORM** est une technique de programmation qui permet de convertir des données entre le système de types d'un langage de programmation orienté objet et celui d'une base de données relationnelle. L'ORM crée une couche d'abstraction qui permet au développeur de manipuler les données sous forme d'objets plutôt que d'écrire des requêtes SQL brutes.

Eloquent, l'ORM intégré à Laravel, permet par exemple de définir un modèle `Subject` qui correspond directement à la table `subjects` de la base de données, avec ses relations (appartient à un étudiant, possède plusieurs fichiers de thèse, etc.).

1.6. Intelligence artificielle générative

L'**intelligence artificielle générative** désigne les systèmes d'IA capables de produire du contenu original (texte, images, code, etc.) à partir de modèles de langage entraînés sur de vastes corpus de données. Les modèles les plus connus incluent GPT (OpenAI), Gemini (Google) et Claude (Anthropic).

Dans le contexte académique, l'émergence de ces outils pose un défi majeur en matière d'intégrité scientifique, car ils permettent potentiellement de générer des textes académiques difficiles à distinguer du travail humain sans outils de détection spécialisés.

1.7. Détection de contenu IA

La **détection de contenu IA** consiste à analyser un texte pour déterminer la probabilité qu'il ait été généré par un modèle d'intelligence artificielle. Les outils de détection, tels que **GPTZero**, utilisent des caractéristiques statistiques du texte (perplexité, entropie, patterns de génération) pour attribuer un score de probabilité.

1.8. Contrôle d'accès basé sur les rôles (RBAC)

Le **RBAC** (Role-Based Access Control) est un modèle de sécurité dans lequel les droits d'accès sont attribués aux utilisateurs en fonction de leur rôle au sein de l'organisation. Chaque rôle définit un ensemble de permissions qui déterminent les actions autorisées dans le système.

Ce modèle est particulièrement adapté aux organisations hiérarchiques comme les universités, où les droits varient significativement selon le statut (étudiant, enseignant, responsable administratif).

1.9. Travail de fin de cycle (TFC)

Le **Travail de Fin de Cycle** est un travail de recherche personnel rédigé par l'étudiant de premier cycle universitaire (licence/baccalauréat) sous la direction d'un enseignant. Il sanctionne la fin de la

formation et vise à démontrer la capacité de l'étudiant à mener une réflexion structurée sur un sujet lié à sa discipline.

Section 2 : Présentation de l'organisme d'accueil

2.1. Historique de l'Université Don Bosco de Lubumbashi

L'**Université Don Bosco de Lubumbashi** (UDBL) est un établissement d'enseignement supérieur situé dans la ville de Lubumbashi, en République Démocratique du Congo. Fondée dans la tradition éducative salésienne, elle porte le nom de Saint Jean Bosco, fondateur de la congrégation des Salésiens de Don Bosco, dont la mission éducative est reconnue mondialement.

L'université s'est progressivement développée pour couvrir plusieurs domaines de formation, allant des sciences informatiques aux sciences économiques, en passant par les sciences humaines et la théologie.

2.2. Structure organisationnelle

L'UDBL est organisée en **quatre facultés**, chacune regroupant une ou plusieurs filières (départements) :

A. Faculté des Sciences Informatiques (ESIS)

- Génie Logiciel (GL)
- Réseaux et Administration Système (AS)
- Design et Multimédia (DSN)
- Management des Systèmes d'Information (MSI)
- Data Science (DS)
- DevOps et Sécurité (DEVOPS)
- Communication Numérique (CN)

B. Faculté des Sciences Économiques (ECOPO)

- Gestion des Entreprises et Ingénierie Financière (GEIF)
- Management Commercial et Marketing (MCM)
- Gestion des Affaires Publiques (GAP)
- Diversification et Développement Agro-Alimentaire (DDAI)

C. Faculté des Sciences Humaines (KANSEBULA)

- Sciences de l'Homme et de la Société (SHS)

D. Faculté de Théologie (THEOLOGICUM)

- Théologie (THEO)

2.3. Mission et vision

L'UDBL a pour mission de former des cadres compétents, responsables et innovants, capables de contribuer au développement socio-économique de la République Démocratique du Congo et de l'Afrique. Sa vision s'articule autour de l'excellence académique, de l'innovation technologique et des valeurs humanistes héritées de la tradition salésienne.

Section 3 : Analyse du système existant

3.1. Description du processus actuel

- Le processus actuel de gestion des TFC à l'Université Don Bosco de Lubumbashi se déroule de la manière suivante :
- Dépôt de la fiche de sujet** : l'étudiant remplit une fiche de proposition de TFC et l'envoie par mail.
 - Attente de la réponse** : l'étudiant attend la décision académique, qui lui est généralement transmise par mail après traitement du dossier.
 - Attribution du directeur** : un enseignant est désigné comme directeur de mémoire par le chef de département.
 - Notification de l'enseignant** : l'enseignant désigné est informé verbalement, par téléphone ou parfois par mail.
 - Rédaction et encadrement** : l'étudiant rédige son travail sous la supervision de son directeur, avec des échanges essentiellement en présentiel.
 - Dépôt du document** : le mémoire achevé est imprimé puis déposé à l'administration pour vérification par le comité de censure. Aucune vérification automatisée de plagiat ou de contenu IA n'est effectuée.
 - Autorisation de soutenance** : le directeur donne son accord pour la soutenance de manière verbale ou par mail.
 - Archivage** : les mémoires sont conservés dans des armoires physiques au secrétariat, avec un accès limité et aucun système de recherche.

3.2. Critique du système existant

Critère	Système actuel	Problème identifié
Support	Mail, papier et communication informelle	Risque de perte, lenteur, absence de traçabilité
Validation des sujets	Traitement manuel des fiches et réponses par mail	Délais importants et faible visibilité sur l'avancement
Notification	Mail, verbal, téléphone	L'information peut être tardive, incomplète ou non historisée

Critère	Système actuel	Problème identifié
Suivi	Registres manuscrits	Aucune vue d'ensemble en temps réel
Détection plagiat/IA	Inexistante lors de la vérification par le comité de censure	Risque d'intégrité académique
Archivage	Physique (armoires)	Accès difficile, risque de détérioration
Communication	Fragmentée	Échanges non documentés

3.3. Solutions proposées

Pour pallier les insuffisances identifiées, nous proposons la conception et la réalisation d'une **plateforme web de gestion des TFC** qui offre :

Problème	Solution proposée
Lenteur de validation	Workflow automatisé avec notifications instantanées
Absence de traçabilité	Journal d'activité complet avec historique des actions
Risque de perte de documents	Stockage numérique sécurisé sur serveur
Pas de détection plagiat/IA	Module d'analyse IA intégré (API GPTZero)
Archives inaccessibles	Archives publiques consultables en ligne avec recherche
Communication fragmentée	Système de notifications par e-mail et in-app
Absence de contrôle d'accès	Système RBAC avec quatre rôles distincts

Section 4 : Choix technologiques

4.1. Technologies retenues

Le choix des technologies s'est porté sur un écosystème robuste, moderne et bien documenté :

Composant	Technologie	Version	Justification
Langage serveur	PHP	8.2+	Langage mature, large communauté, performant
Framework backend	Laravel	12.0	Framework PHP le plus populaire, architecture MVC, Eloquent ORM, système d'authentification intégré
Frontend	Blade + Tailwind CSS	3.x	Moteur de template intégré à Laravel + framework CSS utilitaire moderne

Composant	Technologie	Version	Justification
JavaScript	Alpine.js	3.x	Framework JS léger pour l'interactivité côté client
Base de données	PostgreSQL	14+	SGBD relationnel robuste, adapté aux contraintes transactionnelles et au volume de données du système
ORM	Eloquent	—	ORM intégré à Laravel, approche Active Record
Gestion des rôles	Spatie Laravel Permission	7.0	Package de référence pour le RBAC dans Laravel
Analyse PDF	smalot/pdfparser	2.12	Extraction de texte depuis les fichiers PDF
Détection IA	API GPTZero	v2	Service spécialisé dans la détection de contenu IA
Authentification	Laravel Breeze	—	Scaffolding d'authentification léger et personnalisable
Bundler d'assets	Vite	7.0	Build tool rapide pour CSS/JS
Serveur de dev	PHP Artisan Serve	—	Serveur intégré pour le développement local

4.2. Justification du choix de Laravel

Le choix de **Laravel** comme framework principal se justifie par :

1. **L'architecture MVC native** : Laravel impose une structure claire qui sépare la logique métier, la présentation et le routage, facilitant la maintenance et l'évolution du code.
2. **Eloquent ORM** : permet de définir des modèles qui reflètent directement la structure de la base de données, avec un système de relations (un-à-un, un-à-plusieurs, plusieurs-à-plusieurs) intuitif et puissant.
3. **Le système de migration** : permet de versionner la structure de la base de données et de la reproduire de manière identique sur différents environnements.
4. **L'écosystème riche** : Laravel dispose d'une large collection de packages (Spatie Permission, Breeze, etc.) qui accélèrent considérablement le développement.
5. **Le système de notification** : Laravel intègre nativement un mécanisme de notifications multi-canaux (e-mail, base de données, SMS, etc.) qui répond parfaitement à nos besoins de communication automatisée.
6. **La sécurité intégrée** : protection CSRF, hachage des mots de passe (bcrypt), requêtes préparées contre les injections SQL, validation des entrées.

4.3. Justification du choix de GPTZero

GPTZero a été retenu comme service de détection de contenu IA pour les raisons suivantes :

1. **Spécialisation** : GPTZero est l'un des premiers et des plus reconnus outils de détection de contenu généré par IA ;
 2. **API REST accessible** : l'API est bien documentée et s'intègre facilement dans une application web ;
 3. **Scores détaillés** : l'API fournit des scores de probabilité (contenu complètement généré, partiellement généré) qui permettent une évaluation nuancée ;
 4. **Mode de secours** : notre implémentation prévoit un mode de simulation en cas d'indisponibilité de l'API, garantissant la continuité du service.
-

CHAPITRE DEUXIÈME : CONCEPTION ET MODÉLISATION DU SYSTÈME D'INFORMATION

Section 1 : Identification des acteurs et des besoins

1.1. Acteurs du système

Notre système identifie **quatre acteurs principaux**, chacun disposant d'un rôle et de permissions spécifiques :

Acteur	Rôle	Description
Étudiant	Etudiant	Utilisateur inscrit pouvant soumettre un sujet de TFC, déposer ses fichiers PDF (version jury et version finale) et consulter ses notifications et rapports d'analyse.
Chef de Département	Chef de département	Responsable académique chargé de valider ou rejeter les sujets soumis dans sa filière, d'assigner les directeurs de mémoire et de suivre l'avancement des TFC.
Enseignant	Enseignant	Directeur de mémoire chargé de superviser les travaux qui lui sont assignés, de consulter les rapports d'analyse IA, d'autoriser la soutenance (Feu Vert) et de retirer cette autorisation si nécessaire avant dépôt de la version finale.
Administrateur	Admin	Super-utilisateur chargé de la gestion globale du système : utilisateurs, facultés, filières, années académiques, paramètres et journal d'activité.

Un cinquième acteur, le **visiteur public** (non authentifié), peut consulter les archives des travaux défendus et télécharger les versions finales.

1.2. Besoins fonctionnels

A. Besoins de l'Étudiant

- Disposer d'un compte créé par l'administrateur avec matricule universitaire ;
- Soumettre un sujet de TFC via un formulaire structuré en 5 étapes ;
- Consulter l'état de son sujet (en attente, validé, rejeté) ;
- Déposer la version « jury » de son mémoire (PDF, max 20 Mo) ;
- Consulter le rapport d'analyse IA de son fichier ;
- Recevoir la notification de « Feu Vert » de son encadreur ;
- Recevoir, le cas échéant, la notification de retrait du « Feu Vert » ;
- Consulter la date et la salle de soutenance planifiées ;

- Déposer la version « finale » après autorisation de soutenance ;
- Consulter ses notifications.

B. Besoins du Chef de Département

- Visualiser les sujets en attente de validation dans son département ;
- Valider un sujet et assigner un enseignant encadreur ;
- Rejeter un sujet avec un motif détaillé ;
- Suivre l'ensemble des TFC de son département ;
- Planifier la soutenance (date et salle) après le Feu Vert de l'encadreur ;
- Consulter les rapports d'analyse IA ;
- Exporter la liste des sujets au format CSV.

C. Besoins de l'Enseignant

- Consulter la liste des sujets dont il est l'encadreur ;
- Télécharger les fichiers déposés par ses étudiants ;
- Consulter les rapports d'analyse IA ;
- Autoriser la soutenance (donner le « Feu Vert ») ;
- Retirer le « Feu Vert » tant que la version finale n'est pas encore déposée.

D. Besoins de l'Administrateur

- Gérer les utilisateurs (CRUD, blocage, réinitialisation de mot de passe) ;
- Gérer les facultés et les filières ;
- Gérer les années académiques (créer, activer, clôturer) ;
- Configurer les paramètres du système (seuils IA, dates limites) ;
- Consulter le journal d'activité ;
- Accéder à toutes les fonctionnalités des autres rôles.

E. Besoins du visiteur public

- Consulter les archives des travaux défendus ;
- Rechercher par titre, étudiant, filière, année académique ou type ;
- Télécharger la version finale des travaux archivés.

1.3. Besoins non fonctionnels

- **Performance** : temps de réponse inférieur à 3 secondes pour les opérations courantes ;
- **Sécurité** : authentification obligatoire, protection CSRF, hachage des mots de passe, contrôle d'accès RBAC ;
- **Ergonomie** : interface responsive, intuitive et cohérente, compatible avec les écrans mobiles et desktop ;
- **Fiabilité** : validation des entrées, gestion des erreurs, journalisation des actions ;
- **Maintenabilité** : code structuré selon l'architecture MVC, commenté et versionné ;

- **Extensibilité** : architecture modulaire permettant l'ajout futur de fonctionnalités.

Section 2 : Diagrammes de cas d'utilisation

2.1. Diagramme de cas d'utilisation global

[object Object]

2.2. Description détaillée des cas d'utilisation

Tableau 3 : Description des cas d'utilisation principaux

N°	Cas d'utilisation	Acteur principal	Précondition	Postcondition
CU01	Créer un compte utilisateur	Administrateur	Rôle et filière définis	Compte créé et actif
CU02	Se connecter	Tout utilisateur	Compte existant et non bloqué	Session authentifiée, redirection vers le tableau de bord
CU03	Soumettre un sujet	Étudiant	Connecté, pas de sujet pending/validé	Sujet créé (status=pending notification envoyée au CP
CU04	Valider un sujet	Chef Département	Sujet pending du même département	Status=validated, enseignant assigné,

N°	Cas d'utilisation	Acteur principal	Précondition	Postcondition
				notifications envoyées
CU05	Rejeter un sujet	Chef Département	Sujet pending du même département	Status=rejected, motif enregistré, notification envoyée
CU06	Déposer fichier TFC	Étudiant	Sujet validé (version jury) ou defense_validated (version finale)	Fichier stocké, analyse IA lancée, notification à l'encadreur
CU07	Autoriser soutenance	Enseignant	Est l'encadreur, sujet validé	defense_validated=true, notification « Feu Vert » à l'étudiant
CU08	Consulter archives	Visiteur / Tout utilisateur	Aucune	Affichage des travaux défendus
CU09	Gérer utilisateurs	Administrateur	Connecté, rôle Admin	Utilisateur créé/modifié/bloqué
CU10	Gérer facultés	Administrateur	Connecté, rôle Admin	Faculté créée/modifiée/supprimée

N°	Cas d'utilisation	Acteur principal	Précondition	Postcondition
CU11	Gérer filières	Administrateur	Connecté, rôle Admin	Filière créée/modifiée/supprimée
CU12	Gérer années académiques	Administrateur	Connecté, rôle Admin	Année créée/activée/clôturée
CU13	Planifier soutenance	Chef Département	Sujet avec <code>defense_validated=true</code> du même département	<code>defense_date</code> et <code>defense_room</code> enregistrés
CU14	Retirer autorisation de soutenance	Enseignant	Est l'encadreur, <code>defense_validated=true</code> , version finale non déposée	<code>defense_validated=false</code> , <code>defense_date/defense_roor</code> remis à null, motif de retrait enregistré, notification de retrait envoyée

Section 3 : Diagrammes de séquence

3.1. Diagramme de séquence — Soumission d'un sujet (CU03)

[object Object]

3.2. Diagramme de séquence — Validation d'un sujet (CU04)

[object Object]

3.3. Diagramme de séquence — Dépôt de fichier avec analyse IA (CU06)

[object Object]

3.4. Diagramme de séquence — Autorisation de soutenance (CU07)

[object Object]

Section 4 : Diagramme de classes

4.1. Diagramme de classes principal

[object Object]

4.2. Relations entre les classes

Relation	Type	Cardinalité	Description
Faculty → Department	One-to-Many	1..*	Une faculté contient une ou plusieurs filières
Department → User	One-to-Many	1..*	Une filière contient plusieurs utilisateurs
Department → Subject	One-to-Many	0..*	Une filière peut avoir plusieurs sujets
User → Subject (student)	One-to-Many	0..*	Un étudiant peut avoir plusieurs sujets (historique)
User → Subject (teacher)	One-to-Many	0..*	Un enseignant peut encadrer plusieurs sujets
AcademicYear → Subject	One-to-Many	0..*	Une année académique regroupe plusieurs sujets
Subject → ThesisFile	One-to-Many	0..2	Un sujet peut avoir au maximum 2 fichiers (jury + final)
ThesisFile → AiReport	One-to-One	0..1	Un fichier peut avoir un rapport d'analyse IA
User → ActivityLog	One-to-Many	0..*	Un utilisateur génère plusieurs entrées dans le journal

Section 5 : Modèle relationnel de la base de données

5.1. Schéma relationnel

Le passage du diagramme de classes au schéma relationnel se fait selon les règles classiques de transformation :

```
[object Object]
```

5.2. Dictionnaire de données

Tableau 4 : Dictionnaire de données

Entité	Attribut	Type	Taille	Contrainte	Description
Faculty	id	INT	—	PK, AUTO_INCREMENT	Identifiant unique
Faculty	name	VARCHAR	255	UNIQUE, NOT NULL	Nom de la faculté
Faculty	code	VARCHAR	50	UNIQUE, NOT NULL	Code abrégé (ex: ESIS)
Faculty	description	TEXT	—	NULLABLE	Description détaillée
Department	id	INT	—	PK, AUTO_INCREMENT	Identifiant unique
Department	faculty_id	INT	—	FK → faculties, NOT NULL	Rattachement à une faculté
Department	name	VARCHAR	255	UNIQUE, NOT NULL	Nom de la filière
Department	code	VARCHAR	50	UNIQUE, NOT NULL	Code abrégé (ex: GL)
User	id	INT	—	PK, AUTO_INCREMENT	Identifiant unique
User	name	VARCHAR	255	NOT NULL	Nom complet
User	email	VARCHAR	255	UNIQUE, NOT NULL	Adresse e-mail
User	password	VARCHAR	255	NOT NULL	Mot de passe haché (bcrypt)
User	matricule	VARCHAR	255	UNIQUE, NOT NULL	Matricule universitaire

Entité	Attribut	Type	Taille	Contrainte	Description
User	department_id	INT	—	FK → departments	Filière d'appartenance
User	is_blocked	BOOLEAN	—	DEFAULT false	Indicateur de blocage
Subject	id	INT	—	PK, AUTO_INCREMENT	Identifiant unique
Subject	title	VARCHAR	255	NOT NULL	Titre du sujet
Subject	subject_type	ENUM	—	tfc / memoire	Type de travail
Subject	status	ENUM	—	pending / validated / rejected	État du sujet
Subject	defense_validated	BOOLEAN	—	DEFAULT false	Feu vert pour soutenance
Subject	defense_date	DATETIME	—	NULLABLE	Date et heure planifiées de soutenance
Subject	defense_room	VARCHAR	100	NULLABLE	Salle de soutenance
Subject	student_id	INT	—	FK → users, NOT NULL	Étudiant propriétaire
Subject	teacher_id	INT	—	FK → users, NULLABLE	Enseignant encadreur
ThesisFile	id	INT	—	PK, AUTO_INCREMENT	Identifiant unique
ThesisFile	subject_id	INT	—	FK → subjects	Sujet associé
ThesisFile	file_path	VARCHAR	255	NOT NULL	Chemin du fichier stocké
ThesisFile	version_type	ENUM	—	jury / final	Type de version
AiReport	id	INT	—	PK, AUTO_INCREMENT	Identifiant unique
AiReport	thesis_file_id	INT	—	FK → thesis_files	Fichier analysé
AiReport	similarity_score	INT	—	0-100	Score de similarité (%)

Entité	Attribut	Type	Taille	Contrainte	Description
AiReport	ai_score	INT	—	0-100	Score de contenu IA (%)
AiReport	details	JSON	—	—	Détails de l'analyse

Section 6 : Diagramme d'activités

6.1. Processus complet de gestion d'un TFC

[object Object]

CHAPITRE TROISIÈME : IMPLÉMENTATION ET PRÉSENTATION DES RÉSULTATS

Section 1 : Environnement de développement

1.1. Outils de développement

Outil	Rôle	Version
Visual Studio Code	Éditeur de code principal	Dernière version
Laragon / XAMPP + PostgreSQL	Environnement de développement local (PHP, serveur web, SGBD)	—
Composer	Gestionnaire de dépendances PHP	2.x
Node.js + npm	Gestionnaire de paquets frontend	18+
Git	Système de contrôle de version	2.x
Navigateur web	Tests et validation de l'interface	Chrome / Edge
Artisan	Interface en ligne de commande de Laravel	—

1.2. Configuration requise

Composant	Exigence minimale
PHP	Version 8.2 ou supérieure
Serveur web	Apache / Nginx (ou <code>php artisan serve</code> en développement)
Base de données	PostgreSQL 14+
Mémoire RAM	2 Go minimum
Espace disque	500 Mo pour l'application + espace pour les fichiers PDF

1.3. Installation et déploiement

L'installation de l'application suit les étapes standard d'un projet Laravel :

```
[object Object]
```

Section 2: Architecture technique de l'application

2.1. Structure des répertoires

L'application suit la structure standard de Laravel, organisée selon le patron MVC :

```
[object Object]
```

2.2. Architecture MVC appliquée

L'architecture MVC de notre application se traduit concrètement de la manière suivante :

Modèle (Model) — Les 9 modèles Eloquent (dans `app/Models/`) représentent les entités métier. Chaque modèle définit :

- Les attributs remplissables (`$fillable`) pour la protection contre l'assignation de masse ;
- Les relations avec les autres modèles (`belongsTo` , `hasMany` , `hasOne`) ;
- Les accesseurs et mutateurs pour la transformation des données ;
- Les méthodes métier (ex: `AcademicYear::current()` , `Subject::isValidated()`).

Vue (View) — Les templates Blade (dans `resources/views/`) utilisent :

- Le moteur de template Blade avec ses directives (`@if` , `@foreach` , `@auth` , `@role`) ;
- Tailwind CSS pour le design responsive et cohérent ;
- Alpine.js pour les interactions dynamiques côté client (formulaires multi-étapes, modales) ;
- Un système de layouts pour la réutilisation des structures communes.

Contrôleur (Controller) — Les 12 contrôleurs (dans `app/Http/Controllers/`) orchestrent :

- La réception et la validation des requêtes HTTP ;
- L'appel aux modèles pour la récupération et la manipulation des données ;
- La sélection et le rendu de la vue appropriée ;
- La gestion des autorisations via les rôles et permissions.

2.3. Flux de traitement d'une requête

```
[object Object]
```

1. Le navigateur envoie une requête HTTP ;
2. Le routeur (`routes/web.php`) identifie la route correspondante ;
3. Les middleware s'exécutent (vérification d'authentification, vérification du blocage, vérification du rôle) ;
4. Le contrôleur approprié reçoit la requête ;
5. Le contrôleur interagit avec le(s) modèle(s) Eloquent ;
6. Les modèles interrogent la base de données via l'ORM ;
7. Le contrôleur passe les données à la vue Blade ;
8. La vue génère le HTML et renvoie la réponse au navigateur.

Section 3 : Structure de la base de données

3.1. Tables principales

L'application utilise **22 tables** au total : **10 tables métier** (users, faculties, departments, subjects, thesis_files, ai_reports, academic_years, system_settings, activity_logs, notifications) et **12 tables techniques** (authentification, cache, jobs et permissions RBAC).

Tableau 5 : Structure de la table **users**

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
name	VARCHAR(255)	NOT NULL	Nom complet de l'utilisateur
email	VARCHAR(255)	UNIQUE, NOT NULL	Adresse e-mail (identifiant de connexion)
email_verified_at	TIMESTAMP	NULLABLE	Date de vérification de l'e-mail
password	VARCHAR(255)	NOT NULL	Mot de passe haché (bcrypt)
matricule	VARCHAR(255)	UNIQUE, NOT NULL	Matricule universitaire
department_id	BIGINT UNSIGNED	FK → departments	Filière d'appartenance
is_blocked	BOOLEAN	DEFAULT false	Indicateur de suspension du compte
remember_token	VARCHAR(100)	NULLABLE	Token « Se souvenir de moi »
created_at	TIMESTAMP	—	Date de création
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 6 : Structure de la table **faculties**

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
name	VARCHAR(255)	UNIQUE, NOT NULL	Nom de la faculté
code	VARCHAR(50)	UNIQUE, NOT NULL	Code abrégé (ex: ESIS)
description	TEXT	NULLABLE	Description de la faculté
created_at	TIMESTAMP	—	Date de création
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 7 : Structure de la table **departments**

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
faculty_id	BIGINT UNSIGNED	FK → faculties, NOT NULL	Faculté de rattachement
name	VARCHAR(255)	UNIQUE, NOT NULL	Nom de la filière
code	VARCHAR(50)	UNIQUE, NOT NULL	Code abrégé (ex: GL)
description	TEXT	NULLABLE	Description de la filière
created_at	TIMESTAMP	—	Date de création
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 8 : Structure de la table **subjects**

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
title	VARCHAR(255)	NOT NULL	Titre du sujet de TFC
subject_type	VARCHAR(255)	NULLABLE	Type : « tfc » ou « memoire »
description	TEXT	NOT NULL	Description synthétique
status	ENUM	DEFAULT 'pending'	État : pending / validated / rejected
rejection_reason	TEXT	NULLABLE	Motif de rejet (si rejeté)
defense_validated	BOOLEAN	DEFAULT false	Autorisation de soutenance
defense_date	DATETIME	NULLABLE	Date et heure de soutenance planifiées
defense_room	VARCHAR(100)	NULLABLE	Salle de soutenance planifiée
context_relevance	TEXT	NULLABLE	Contexte et pertinence du sujet
challenges	TEXT	NULLABLE	Problématique identifiée
research_question	TEXT	NULLABLE	Question de recherche
hypothesis	TEXT	NULLABLE	Hypothèses formulées
general_objective	TEXT	NULLABLE	Objectif général

Colonne	Type	Contrainte	Description
specific_objectives	JSON	NULLABLE	Liste des objectifs spécifiques
state_of_art	JSON	NULLABLE	Revue de littérature structurée
demarcations	TEXT	NULLABLE	Délimitation du sujet
methodologies	TEXT	NULLABLE	Méthodologies retenues
student_id	BIGINT UNSIGNED	FK → users, NOT NULL	Étudiant auteur
teacher_id	BIGINT UNSIGNED	FK → users, NULLABLE	Enseignant encadreur
department_id	BIGINT UNSIGNED	FK → departments	Filière
academic_year_id	BIGINT UNSIGNED	FK → academic_years, NULLABLE	Année académique
created_at	TIMESTAMP	—	Date de soumission
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 9 : Structure de la table `thesis_files`

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
subject_id	BIGINT UNSIGNED	FK → subjects	Sujet associé
file_path	VARCHAR(255)	NOT NULL	Chemin de stockage du fichier
original_name	VARCHAR(255)	NOT NULL	Nom original du fichier uploadé
version_type	ENUM	NOT NULL	Type : « jury » ou « final »
created_at	TIMESTAMP	—	Date de dépôt
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 10 : Structure de la table `ai_reports`

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique

Colonne	Type	Contrainte	Description
thesis_file_id	BIGINT UNSIGNED	FK → thesis_files	Fichier analysé
similarity_score	INTEGER	NOT NULL (0-100)	Score de similarité détectée (%)
ai_score	INTEGER	NOT NULL (0-100)	Score de contenu IA détecté (%)
details	JSON	NULLABLE	Détails de l'analyse (pages signalées, métadonnées)
created_at	TIMESTAMP	—	Date de l'analyse
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 11 : Structure de la table **academic_years**

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
name	VARCHAR(255)	UNIQUE, NOT NULL	Libellé (ex: « 2025-2026 »)
start_date	DATE	NOT NULL	Date de début
end_date	DATE	NOT NULL	Date de fin
is_current	BOOLEAN	DEFAULT false	Année en cours
is_closed	BOOLEAN	DEFAULT false	Année clôturée
created_at	TIMESTAMP	—	Date de création
updated_at	TIMESTAMP	—	Date de dernière modification

Tableau 12 : Structure de la table **activity_logs**

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
user_id	BIGINT UNSIGNED	FK → users, NULLABLE	Utilisateur ayant effectué l'action
action	VARCHAR(255)	NOT NULL	Type d'action (created, updated, deleted, etc.)
model_type	VARCHAR(255)	NULLABLE	Classe du modèle concerné

Colonne	Type	Contrainte	Description
model_id	BIGINT UNSIGNED	NULLABLE	ID de l'enregistrement concerné
description	VARCHAR(255)	NOT NULL	Description lisible de l'action
old_values	JSON	NULLABLE	Valeurs avant modification
new_values	JSON	NULLABLE	Valeurs après modification
created_at	TIMESTAMP	—	Date et heure de l'action
updated_at	TIMESTAMP	—	—

Tableau 13 : Structure de la table `system_settings`

Colonne	Type	Contrainte	Description
id	BIGINT UNSIGNED	PK, AI	Identifiant unique
key	VARCHAR(255)	UNIQUE, NOT NULL	Clé du paramètre
value	TEXT	NULLABLE	Valeur du paramètre
type	VARCHAR(255)	NOT NULL	Type de donnée (string, integer, boolean, date)
group	VARCHAR(255)	NOT NULL	Groupe de paramètres (general, deadlines, ai)
label	VARCHAR(255)	NOT NULL	Libellé lisible
description	TEXT	NULLABLE	Description du paramètre
created_at	TIMESTAMP	—	—
updated_at	TIMESTAMP	—	—

Section 4 : Présentation des interfaces

4.1. Page d'accueil (welcome)

La page d'accueil constitue le point d'entrée public de l'application. Elle présente :

- Une **barre institutionnelle** affichant le nom de l'université et l'année académique ;
- Un **en-tête principal** avec le logo de l'UDBL, le titre de la plateforme et deux boutons d'action (Connexion et Archives) ;
- Une section « **Accès selon votre profil** » décrivant les quatre rôles avec des icônes distinctives ;
- Une section « **Procédure de dépôt de TFC** » présentant les quatre étapes du processus ;

- Une section « **Vérification par analyse IA** » expliquant le module de détection avec une grille d'interprétation des scores ;
- Un **encadré d'informations pratiques** ;
- Un **pied de page** avec les coordonnées de l'université et la liste des facultés.

[Figure 15 : Capture d'écran — Page d'accueil]

4.2. Page de connexion

Le formulaire de connexion demande l'adresse e-mail et le mot de passe. Un lien vers la réinitialisation du mot de passe est disponible. Le système vérifie automatiquement si le compte est bloqué après authentification.

[Figure 16 : Capture d'écran — Formulaire de connexion]

4.3. Tableau de bord Administrateur

Le tableau de bord administrateur affiche :

- Des **compteurs en temps réel** : total utilisateurs, total sujets, sujets en attente/validés/rejetés, nombre de filières, nombre de facultés, comptes bloqués ;
- Un **graphique de répartition** des sujets par statut ;
- Un **graphique par département** montrant la distribution des TFC ;
- L'**année académique courante** ;
- Les **10 dernières entrées** du journal d'activité ;
- Les **5 derniers utilisateurs** inscrits.

[Figure 17 : Capture d'écran — Tableau de bord Administrateur]

4.4. Tableau de bord Étudiant

Le tableau de bord étudiant présente :

- L'**état actuel** de son sujet (en attente, validé, rejeté, ou invitation à soumettre) ;
- Le **nom de l'encadreur assigné** (si le sujet est validé) ;
- La **possibilité de déposer un fichier** (version jury ou finale selon l'état) ;
- Le **rapport d'analyse IA** (scores et détails) si un fichier a été analysé ;
- Le **statut du Feu Vert** (autorisation de soutenance) ;
- La **date et la salle de soutenance** lorsqu'elles sont planifiées.

[Figure 18 : Capture d'écran — Tableau de bord Étudiant]

4.5. Formulaire de soumission de sujet (5 étapes)

Le formulaire est structuré en **cinq étapes progressives** avec Alpine.js pour la navigation :

Étape 1 — Informations générales

- Type de travail (TFC ou Mémoire) ;
- Titre du sujet.

Étape 2 — Contexte et problématique

- Contexte et pertinence ;
- Problématique identifiée ;
- Question de recherche.

Étape 3 — Objectifs et hypothèses

- Objectif général ;
- Objectifs spécifiques (ajout dynamique) ;
- Hypothèse de recherche.

Étape 4 — Revue de littérature

- Entrées structurées : auteur, année, titre, contribution (ajout dynamique).

Étape 5 — Méthodologie

- Délimitation du sujet ;
- Méthodologies retenues.

[Figure 19 : Capture d'écran — Formulaire de soumission de sujet]

4.6. Tableau de bord Chef de Département

Ce tableau de bord affiche :

- Le **nombre de sujets en attente** de validation dans son département ;
- La **liste des sujets** avec les actions possibles (valider, rejeter, assigner) ;
- L'**action de planification** (date et salle) pour les sujets ayant reçu le Feu Vert ;
- La **liste des enseignants** du département comme directeurs potentiels ;
- Un accès aux **rapports IA** des fichiers déposés.

[Figure 20 : Capture d'écran — Tableau de bord Chef de Département]

4.7. Tableau de bord Enseignant

Le tableau de bord enseignant affiche :

- La **liste des sujets** dont il est l'encadreur ;
- Pour chaque sujet : l'étudiant, le statut, les fichiers déposés, les rapports IA ;
- L'**affichage de la planification** (date/salle) lorsqu'elle est définie par le Chef de Département ;
- Le **bouton « Autoriser la soutenance »** (Feu Vert) lorsque les conditions sont remplies.

[Figure 21 : Capture d'écran — Tableau de bord Enseignant]

4.8. Rapport d'analyse IA

L'interface d'affichage du rapport IA présente :

- Le **score de contenu IA** avec une barre de progression colorée ;
- Le **score de similarité** avec une barre de progression colorée ;
- La **grille d'interprétation** :
 - Vert (< 20%) : Risque faible — contenu considéré comme original ;
 - Jaune (20–50%) : Risque modéré — vérification recommandée ;
 - Rouge (> 50%) : Risque élevé — entretien avec l'étudiant requis ;
- Les **détails de l'analyse** : date, nombre de pages, nombre de mots, sections signalées.

[Figure 22 : Capture d'écran — Rapport d'analyse IA]

4.9. Archives publiques

La page d'archives est accessible sans authentification et présente :

- Un **formulaire de recherche** avec filtres (recherche textuelle, filière, année académique, type de travail) ;
- Un **tableau paginé** des travaux défendus avec les colonnes : N°, Titre, Étudiant, Directeur, Filière, Année, Type, Télécharger ;
- Un **bouton de téléchargement PDF** pour les versions finales disponibles.

[Figure 23 : Capture d'écran — Archives publiques]

4.10. Gestion des utilisateurs (Admin)

L'interface d'administration des utilisateurs offre :

- Une **liste paginée** avec filtres (recherche, rôle, filière, statut) ;
- Des **actions par utilisateur** : modifier, bloquer/débloquer, réinitialiser le mot de passe, supprimer ;
- Un **formulaire de création** : nom, e-mail, matricule, mot de passe, rôle, filière.

[Figure 24 : Capture d'écran — Gestion des utilisateurs]

4.11. Gestion des facultés et filières (Admin)

L'interface permet :

- La **gestion des facultés** : créer, modifier, supprimer (avec protection contre la suppression de facultés contenant des filières) ;
- La **gestion des filières** : créer avec rattachement à une faculté, modifier, supprimer (avec protection contre la suppression de filières contenant des utilisateurs ou des sujets).

[Figure 25 : Capture d'écran — Gestion des facultés et filières]

Section 5: Sécurité et contrôle d'accès

5.1. Authentification

L'authentification est gérée par **Laravel Breeze**, qui fournit :

- Un formulaire de connexion avec validation ;
- Le hachage des mots de passe via **bcrypt** ;
- La gestion des sessions avec régénération du token après connexion ;
- La vérification de l'adresse e-mail ;
- La réinitialisation de mot de passe par e-mail.

5.2. Contrôle d'accès RBAC

Le contrôle d'accès repose sur le package **Spatie Laravel Permission** qui implémente le modèle RBAC :

Tableau 14 : Matrice rôles-permissions

Permission	Admin	Chef Dept.	Enseignant	Étudiant
subjects.create	—	—	—	✓
subjects.view	✓	✓	✓	✓
subjects.validate	✓	✓	—	—
subjects.reject	✓	✓	—	—
subjects.assign-teacher	✓	✓	—	—
thesis.upload	—	—	—	✓
thesis.download	✓	✓	✓	—
thesis.view-reports	✓	✓	✓	—
thesis.final-deposit	—	—	—	✓
thesis.validate-defense	—	—	✓	—
users.manage	✓	—	—	—
users.block	✓	—	—	—
users.reset-password	✓	—	—	—
departments.manage	✓	—	—	—
academic-years.manage	✓	—	—	—
settings.manage	✓	—	—	—
logs.view	✓	—	—	—

5.3. Middleware de sécurité

Deux middleware personnalisés renforcent la sécurité :

- a) **CheckUserBlocked** : vérifie à chaque requête si le compte de l'utilisateur authentifié a été suspendu. Si oui, la session est immédiatement invalidée et l'utilisateur est redirigé vers la page de connexion avec un message d'erreur.
- b) **EnsureRole** : vérifie que l'utilisateur possède l'un des rôles requis pour accéder à une route. Exemple d'utilisation dans les routes :

```
[object Object]
```

5.4. Protections intégrées de Laravel

- **Protection CSRF** : chaque formulaire inclut un token CSRF vérifié automatiquement ;
- **Requêtes préparées** : Eloquent ORM utilise des requêtes préparées pour prévenir les injections SQL ;
- **Validation des entrées** : toutes les données utilisateur sont validées côté serveur avant traitement ;
- **Protection contre l'assignation de masse** : seuls les attributs listés dans `$fillable` peuvent être assignés en masse ;
- **Stockage sécurisé des fichiers** : les fichiers PDF sont stockés dans le dossier `storage` et servis via un contrôleur qui vérifie les autorisations d'accès.

Section 6 : Tests et validation

6.1. Tests fonctionnels

Les tests fonctionnels ont couvert les scénarios suivants :

Tableau 17 : Résultats des tests fonctionnels

N°	Scénario de test	Résultat attendu	Résultat obtenu	Statut
T01	Création d'un compte étudiant par l'admin	Compte créé, rôle Étudiant assigné	Compte créé avec succès	✓ Réussi
T02	Connexion avec identifiants valides	Redirection vers le dashboard	Dashboard affiché correctement	✓ Réussi
T03	Connexion avec compte bloqué	Rejet + message d'erreur	Session invalidée, message affiché	✓ Réussi
T04	Soumission de sujet (5 étapes)	Sujet créé, notification envoyée	Sujet en status « pending », CP notifié	✓ Réussi
T05	Soumission d'un 2e sujet (déjà pending)	Rejet avec message d'erreur	Message « Vous avez déjà un sujet... »	✓ Réussi

N°	Scénario de test	Résultat attendu	Résultat obtenu	Statut
T06	Validation de sujet par le CP	Status validated, enseignant assigné	Notifications envoyées aux deux parties	✓ Réussi
T07	Rejet de sujet par le CP	Status rejected, motif enregistré	Étudiant notifié avec le motif	✓ Réussi
T08	Upload version jury (sujet validé)	Fichier stocké, analyse IA lancée	Rapport IA généré, enseignant notifié	✓ Réussi
T09	Upload version finale (sans Feu Vert)	Rejet	Message d'erreur approprié	✓ Réussi
T10	Upload version finale (avec Feu Vert)	Fichier stocké, analyse IA lancée	Version finale déposée avec succès	✓ Réussi
T11	Autorisation de soutenance	defense_validated = true	Notification « Feu Vert » envoyée	✓ Réussi
T12	Planification de soutenance (CP)	defense_date + defense_room enregistrés (après Feu Vert)	Date/salle visibles sur les dashboards concernés	✓ Réussi
T13	Consultation des archives (public)	Liste des travaux défendus	Tableau paginé avec filtres fonctionnels	✓ Réussi
T14	Téléchargement version finale (archives)	Fichier PDF téléchargé	Téléchargement effectué	✓ Réussi
T15	Accès admin sans rôle Admin	Erreur 403	Page « Accès non autorisé »	✓ Réussi
T16	Création d'un utilisateur (Admin)	Utilisateur créé, rôle assigné	Log d'activité enregistré	✓ Réussi
T17	Blocage d'un utilisateur	is_blocked = true	Connexion impossible pour l'utilisateur	✓ Réussi
T18	Réinitialisation de mot de passe	Nouveau mot de passe généré	Mot de passe affiché à l'admin	✓ Réussi
T19	Clôture d'une année académique	is_closed = true	TFC archivés, année verrouillée	✓ Réussi
T20	Export CSV des sujets	Fichier CSV téléchargé	Format correct avec encodage UTF-8 BOM	✓ Réussi
T21	Analyse IA avec scores élevés	Alerte visuelle (rouge)	Badge rouge affiché, sections signalées	✓ Réussi
T22	Retrait du Feu Vert (sans version finale)	defense_validated = false + motif obligatoire +	Retrait effectué, motif enregistré, date/salle	✓ Réussi

N°	Scénario de test	Résultat attendu	Résultat obtenu	Statut
		notification de retrait	effacées	

6.2. Grille d'interprétation des scores IA

Tableau 15 : Grille d'interprétation des scores IA

Score IA	Niveau de risque	Couleur	Action recommandée
< 20%	Faible	 Vert	Contenu considéré comme original. Aucune action requise.
20% – 50%	Modéré	 Jaune	Vérification recommandée. L'enseignant devrait examiner les sections signalées.
> 50%	Élevé	 Rouge	Entretien avec l'étudiant requis. Suspicion forte de contenu généré par IA.

6.3. Module de détection IA — Fonctionnement technique

Le service `AiDetectionService` implémente la détection de contenu IA selon le processus suivant :

- Extraction du texte** : le contenu textuel est extrait du fichier PDF à l'aide de la bibliothèque `smalot/pdfparser` ;
 - Appel à l'API** : si une clé API GPTZero est configurée, le texte (limité aux 50 000 premiers caractères) est envoyé à l'API GPTZero via une requête POST ;
 - Analyse des résultats** : l'API retourne des probabilités (`completely_generated_prob` , `average_generated_prob`) qui sont converties en un score de 0 à 100 ;
 - Mode de secours** : en l'absence de clé API, le système active un mode de simulation qui génère des scores aléatoires (à des fins de démonstration et de test) ;
 - Rapport** : un enregistrement `AiReport` est créé avec les scores et les détails de l'analyse.
-

CONCLUSION GÉNÉRALE

Au terme de ce travail de fin de cycle, nous pouvons affirmer que les objectifs fixés ont été atteints. La plateforme **TFC Manager** développée pour l'Université Don Bosco de Lubumbashi constitue une solution fonctionnelle et complète pour la gestion numérique des Travaux de Fin de Cycle.

Réalisations

Les principales réalisations de ce projet sont :

1. **La numérisation complète du processus de gestion des TFC** : depuis la soumission du sujet par l'étudiant jusqu'à la planification de la soutenance et l'archivage du mémoire défendu, chaque étape est désormais dématérialisée et tracée.
2. **L'implémentation d'un système de rôles et permissions** : quatre profils utilisateurs (Administrateur, Chef de Département, Enseignant, Étudiant) disposent chacun d'un espace personnalisé et de fonctionnalités adaptées à leurs responsabilités.
3. **L'intégration d'un module de détection de contenu IA** : chaque mémoire déposé est automatiquement analysé par l'API GPTZero, avec génération d'un rapport détaillé incluant un score de contenu généré par IA et un score de similarité.
4. **La mise en place d'un système de notifications multi-canaux** : six types de notifications (e-mail et in-app) assurent la communication fluide entre tous les acteurs du processus.
5. **La création d'archives publiques consultables** : les travaux défendus sont référencés dans un espace public avec possibilité de recherche et de téléchargement des versions finales.
6. **Un journal d'activité complet** : toutes les actions significatives sont enregistrées avec horodatage, permettant un audit complet du système.
7. **L'ajout d'un module de planification de soutenance** : après autorisation de l'encadreur, le Chef de Département peut définir la date/heure et la salle, visibles par les acteurs concernés.

Vérification des hypothèses

- **Hypothèse 1** (plateforme centralisée) : **confirmée**. L'application offre un point d'accès unique à tous les acteurs, avec des tableaux de bord adaptés à chaque rôle et un workflow automatisé allant de la soumission à la planification de la soutenance.
- **Hypothèse 2** (module de détection IA) : **confirmée**. Le service `AiDetectionService` intégré analyse automatiquement chaque fichier déposé et produit un rapport avec des scores quantitatifs, permettant aux enseignants de prendre des décisions éclairées.
- **Hypothèse 3** (notifications automatisées) : **confirmée**. Les six classes de notifications implémentées couvrent l'ensemble des étapes du workflow et sont générées automatiquement à chaque transition d'état.

- **Hypothèse 4** (archives publiques) : **confirmée**. La page d'archives permet à tout visiteur de consulter les travaux défendus avec un système de recherche multicritères et de télécharger les versions finales.

Limites et perspectives

Malgré les résultats satisfaisants obtenus, certaines limites méritent d'être relevées :

- **La dépendance à une API externe** (GPTZero) pour la détection de contenu IA, avec un mode de simulation en cas d'indisponibilité ;
- **L'absence de gestion complète des sessions de soutenance** (composition du jury, attribution des notes, procès-verbal) ;
- **L'absence de système de messagerie intégrée** entre étudiants et encadreurs pour le suivi de la rédaction.

En termes de perspectives d'amélioration, nous envisageons :

- L'extension du **module de soutenance** vers la gestion du jury, la notation et l'édition d'un procès-verbal ;
- L'intégration d'un **système de messagerie instantanée** entre étudiants et encadreurs ;
- Le développement d'un **module de détection de plagiat** comparant les travaux entre eux (au-delà de la détection IA) ;
- La mise en place d'une **application mobile** complémentaire pour les notifications en temps réel ;
- L'extension du système à d'autres processus académiques (stages, rapports de recherche).

En définitive, ce travail démontre que les technologies web modernes, conjuguées à l'intelligence artificielle, offrent des solutions puissantes pour la modernisation de la gestion académique dans les universités congolaises. La plateforme TFC Manager, dans sa version actuelle, pose les bases d'un outil évolutif au service de l'intégrité académique et de l'efficacité administrative.

BIBLIOGRAPHIE

Ouvrages

1. REIX, R. (2004). *Systèmes d'information et management des organisations*. 5e édition. Paris : Vuibert.
2. SOMMERVILLE, I. (2015). *Software Engineering*. 10th edition. Boston : Pearson.
3. GAMMA, E., HELM, R., JOHNSON, R., VLISSIDES, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading : Addison-Wesley.
4. FOWLER, M. (2003). *Patterns of Enterprise Application Architecture*. Boston : Addison-Wesley.
5. PILONE, D., PITMAN, N. (2005). *UML 2.0 in a Nutshell*. Sebastopol : O'Reilly Media.

Documentation technique

6. LARAVEL (2025). *Laravel 12.x Documentation*. [En ligne]. Disponible sur : <https://laravel.com/docs/12.x>
7. SPATIE (2025). *Laravel Permission Documentation*. [En ligne]. Disponible sur : <https://spatie.be/docs/laravel-permission>
8. TAILWIND CSS (2025). *Tailwind CSS Documentation*. [En ligne]. Disponible sur : <https://tailwindcss.com/docs>
9. GPTZERO (2025). *GPTZero API Documentation*. [En ligne]. Disponible sur : <https://gptzero.me/docs>
10. PHP (2025). *PHP 8.2 Manual*. [En ligne]. Disponible sur : <https://www.php.net/manual/>

Articles et rapports

11. MITCHELL, E., LEE, Y., KHAZANCHI, A. (2023). « DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature ». *Proceedings of the 40th International Conference on Machine Learning*.
 12. KIRCHNER, J. H., AHMAD, L., AARONSON, S., LEIKE, J. (2023). « New AI classifier for indicating AI-written text ». *OpenAI Blog*.
-

ANNEXES — DIAGRAMMES (par figure)

Diagrammes en Noir et Blanc - Version par Figure

Ce document presente les diagrammes figure par figure, dans l'ordre de la liste des figures. Tous les diagrammes sont en noir et blanc.

Figure 2 - Diagramme de cas d'utilisation global

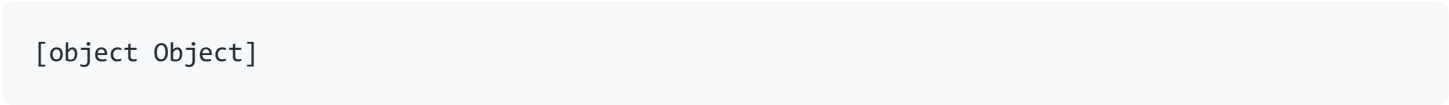


Figure 3 - Cas d'utilisation Etudiant



Figure 4 - Cas d'utilisation Chef de Departement

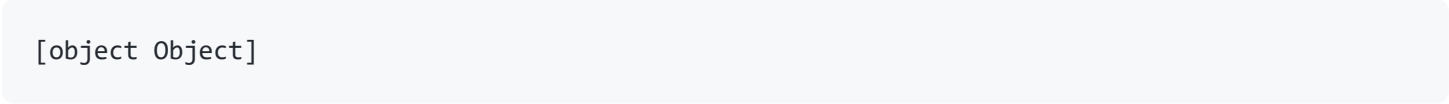


Figure 5 - Cas d'utilisation Enseignant

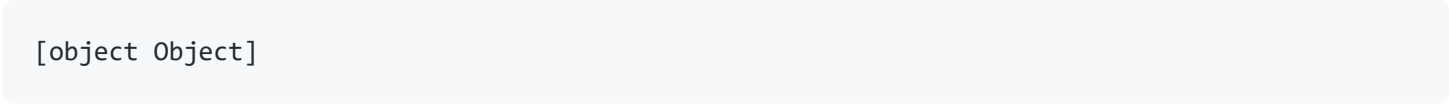


Figure 6 - Cas d'utilisation Administrateur



Figure 7 - Sequence Soumission de sujet

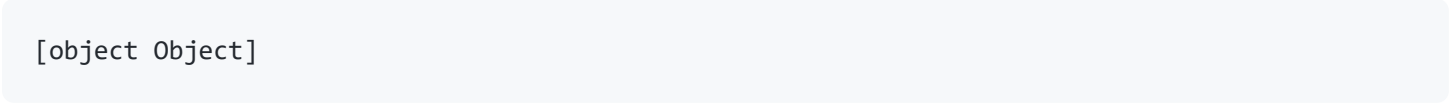


Figure 8 - Sequence Validation ou rejet

[object Object]

Figure 9 - Sequence Depot de fichier avec analyse IA

[object Object]

Figure 10 - Sequence Autorisation de soutenance

[object Object]

Figure 11 - Diagramme de classes

[object Object]

Figure 12 - Modele relationnel de la base de donnees

[object Object]

Figure 13 - Diagramme d'activites du processus complet

[object Object]

Figure 14 - Architecture MVC de Laravel

[object Object]

Recommandation d'usage

- Utiliser `DIAGRAMMES-PAR-CHAPITRE-NB.md` pour la redaction du memoire.
- Utiliser `DIAGRAMMES-PAR-FIGURE-NB.md` pour la liste des figures et annexes.

ANNEXES — DIAGRAMMES (par chapitre)

Diagrammes en Noir et Blanc - Version par Chapitre

Ce document regroupe les diagrammes par chapitre du mémoire. Tous les diagrammes sont en style noir et blanc (fond blanc, traits noirs, texte noir).

Chapitre II - Section 2 : Diagrammes de cas d'utilisation

2.1 Diagramme de cas d'utilisation global

[object Object]

2.2 Diagramme de cas d'utilisation - Etudiant

[object Object]

2.3 Diagramme de cas d'utilisation - Chef de Departement

[object Object]

2.4 Diagramme de cas d'utilisation - Enseignant

[object Object]

2.5 Diagramme de cas d'utilisation - Administrateur

[object Object]

Chapitre II - Section 3 : Diagrammes de sequence

3.1 Sequence - Soumission de sujet

[object Object]

3.2 Sequence - Validation ou rejet de sujet

[object Object]

3.3 Sequence - Depot de fichier TFC avec analyse IA

[object Object]

3.4 Sequence - Autorisation de soutenance

[object Object]

Chapitre II - Section 4 : Diagramme de classes

[object Object]

Chapitre II - Section 5 : Modele relationnel de la base de donnees

[object Object]

Chapitre II - Section 6 : Diagramme d'activites (processus complet)

[object Object]

Chapitre III - Section 2 : Architecture MVC de Laravel

[object Object]

Notes d'impression

- Theme monochrome applique a chaque diagramme.
- Impression recommandee en niveaux de gris.
- Compatible avec export PDF depuis VS Code ou GitHub.